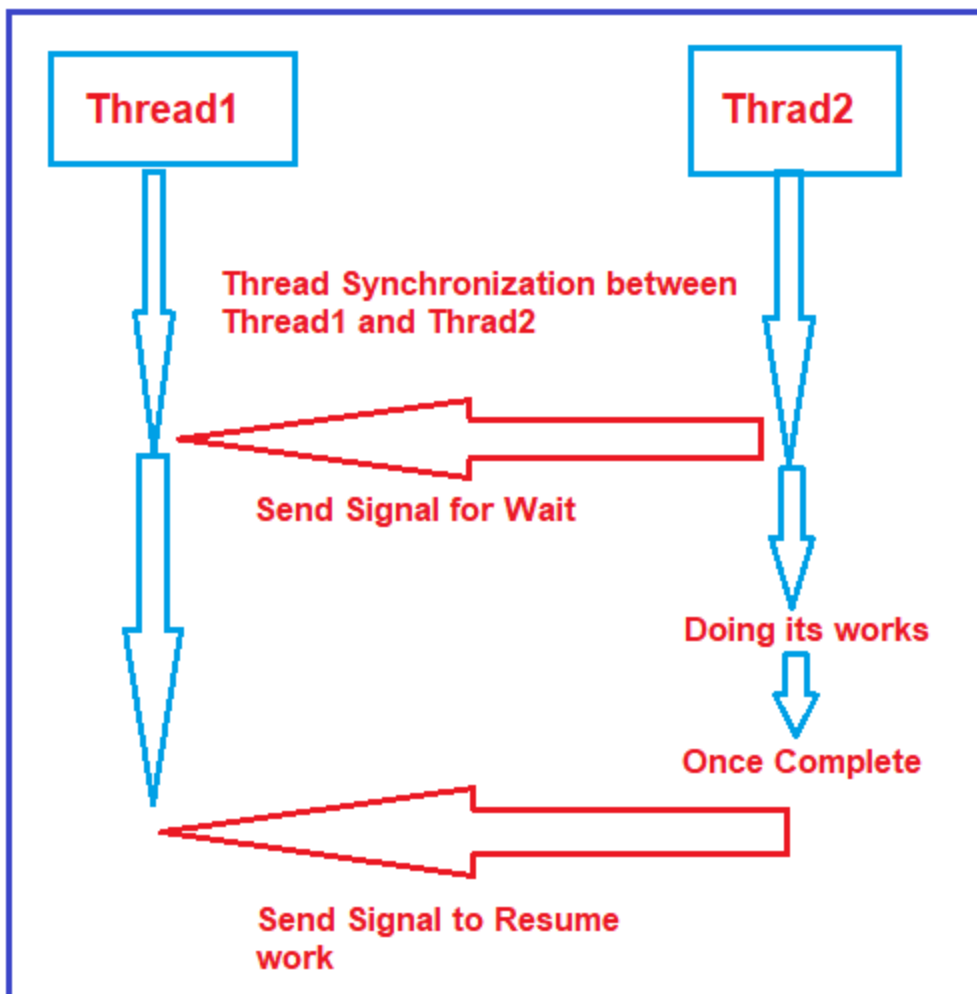


AutoResetEvent and ManualResetEvent in C# with Examples

In this article, I am going to discuss two important concepts of threading i.e. **AutoResetEvent** and **ManualResetEvent** in C# with Examples. In our previous articles, we already saw how to implement thread synchronization using **Lock**, **Monitor**, **Mutex**, **Semaphore**, etc. And there is another way of doing thread synchronization i.e. by using signaling methodology. And both AutoResetEvent and ManualResetEvent in C# help us to implement thread synchronization using signaling methodology.

What is Signaling Methodology?

Let us first understand what is Signalling Methodology and then we will understand how to implement the Signalling Methodology using AutoResetEvent and ManualResetEvent in C#. Let us understand this with an example. Please have a look at the below image. Let's say we have two threads Thread1 and Thread2. And we need to implement thread synchronization between these two threads. For thread synchronization, here thread2 sends a signal to thread1 saying that please go under Wait mode. And then thread2 continues doing its work. And once thread2 finishes its work, again it sends a signal to thread1 saying that you can resume your work from where you halted.



So, in this way, by using the Signalling Methodology we can implement thread synchronization between multiple threads in C#. And both AutoResetEvent and ManualResetEvent in C# help us to achieve this. So, here, first, we will see an example using AutoResetEvent, then we will see an example using ManualResetEvent, and finally, we will see the differences between them.

AutoResetEvent Class in C#:

AutoResetEvent is used to send signals between two threads. This class Notifies a waiting thread that an event has occurred. If you go to the definition of AutoResetEvent class, then you will see the following. This is a sealed class and hence cannot be inherited. And it is inherited from the EventWaitHandle class.

```
namespace System.Threading
{
    ...public sealed class AutoResetEvent : EventWaitHandle
    {
        ...public AutoResetEvent(bool initialState);
    }
}
```

This class provides the following constructor that we can use to create an instance of the AutoResetEvent class in C#.

1. **AutoResetEvent(bool initialState):** It initializes a new instance of the AutoResetEvent class with a Boolean value indicating whether to set the initial state to signaled. Here, if the parameter initialState is true then it set the initial state to signaled; false to set the initial state to non-signaled.

The AutoResetEvent is inherited from the EventWaitHandle class and if you go to the definition of the EventWaitHandle class, then you will see that this EventWaitHandle class implements the WaitHandle class as shown in the below image, and the EventWaitHandle class also has the Set and Reset method that we are going to use with AutoResetEvent object.

```
namespace System.Threading
{
    ...public class EventWaitHandle : WaitHandle
    {
        ...public EventWaitHandle(bool initialState, EventResetMode mode);
        ...public EventWaitHandle(bool initialState, EventResetMode mode, string name);
        ...public EventWaitHandle(bool initialState, EventResetMode mode, string name, out bool createdNew);
        ...public EventWaitHandle(bool initialState, EventResetMode mode, string name, out bool createdNew,
        ...public static EventWaitHandle OpenExisting(string name);
        ...public static EventWaitHandle OpenExisting(string name, EventWaitHandleRights rights);
        ...public static bool TryOpenExisting(string name, out EventWaitHandle result);
        ...public static bool TryOpenExisting(string name, EventWaitHandleRights rights, out EventWaitHandle result);
        ...public EventWaitHandleSecurity GetAccessControl();
        ...public bool Reset();
        ...public bool Set();
        ...public void SetAccessControl(EventWaitHandleSecurity eventSecurity);
    }
}
```

The following two methods of this class, we are going to use in our example.

1. **Set():** This method is used to set the state of the event to signaled, allowing one or more waiting threads to proceed. It returns true if the operation succeeds; otherwise, false.

2. **Reset()**: This method is used to set the state of the event to non signaled, causing threads to block. It returns true if the operation succeeds; otherwise, false.

Again, the `EventWaitHandle` class is inherited from the `WaitHandle` and if you go to the definition of `WaitHandle` class, then you will see that it is an abstract class and this class has some overloaded versions of the `WaitOne` method as shown in the below image. The `WaitOne` method we are going to use with the `AutoResetEvent` object.

```
namespace System.Threading
{
    ...public abstract class WaitHandle : MarshalByRefObject, IDisposable
    {
        ...public const int WaitTimeout = 258;
        ...protected static readonly IntPtr InvalidHandle;

        ...protected WaitHandle();

        ...public virtual IntPtr Handle { get; set; }
        ...public SafeWaitHandle SafeWaitHandle { get; set; }

        ...public static bool SignalAndWait(WaitHandle toSignal, WaitHandle toWaitOn, int millisecondsTimeout, bool exitContext);
        ...public static bool SignalAndWait(WaitHandle toSignal, WaitHandle toWaitOn, TimeSpan timeout, bool exitContext);
        ...public static bool SignalAndWait(WaitHandle toSignal, WaitHandle toWaitOn);
        ...public static bool WaitAll(WaitHandle[] waitHandles, TimeSpan timeout);
        ...public static bool WaitAll(WaitHandle[] waitHandles, int millisecondsTimeout, bool exitContext);
        ...public static bool WaitAll(WaitHandle[] waitHandles, TimeSpan timeout, bool exitContext);
        ...public static bool WaitAll(WaitHandle[] waitHandles);
        ...public static bool WaitAll(WaitHandle[] waitHandles, int millisecondsTimeout);
        ...public static int WaitAny(WaitHandle[] waitHandles, int millisecondsTimeout);
        ...public static int WaitAny(WaitHandle[] waitHandles);
        ...public static int WaitAny(WaitHandle[] waitHandles, TimeSpan timeout);
        ...public static int WaitAny(WaitHandle[] waitHandles, TimeSpan timeout, bool exitContext);
        ...public static int WaitAny(WaitHandle[] waitHandles, int millisecondsTimeout, bool exitContext);
        ...public virtual void Close();
        ...public void Dispose();
        ...public virtual bool WaitOne(int millisecondsTimeout, bool exitContext);
        ...public virtual bool WaitOne(TimeSpan timeout);
        ...public virtual bool WaitOne(int millisecondsTimeout);
        ...public virtual bool WaitOne(TimeSpan timeout, bool exitContext);
        ...public virtual bool WaitOne();
        ...protected virtual void Dispose(bool explicitDisposing);
    }
}
```

We are going to use the following method in our example.

1. **WaitOne()**: The `WaitOne()` method blocks the current thread until the current `WaitHandle` receives a signal. It returns true if the current instance receives a signal. If the current instance is never signaled, `WaitHandle.WaitOne(System.Int32, System.Boolean)` never returns.

How AutoResetEvent Work in C#?

The `AutoResetEvent` in C# maintains a boolean variable in memory. If the boolean variable is false then it blocks the thread and if the boolean variable is true it unblocks the thread. So, when we create an instance of `AutoResetEvent` class, we need to pass the default value of the boolean value to the constructor of `AutoResetEvent` class. The following is the syntax to instantiate an `AutoResetEvent` object.

```
AutoResetEvent autoResetEvent = new AutoResetEvent(false);
```

WaitOne Method

The WaitOne method blocks the current thread and waits for the signal by another thread. That means the WaitOne method puts the current thread into a Sleep state of the thread. The WaitOne method returns true if it receives the signal else returns false. We need to call the WaitOne method on the AutoResetEvent object as follows.

```
autoResetEvent.WaitOne();
```

Another overloaded version of the WaitOne method takes seconds as a parameter and will wait for the specified number of seconds. If it does not get any signal, then the thread will continue its work. Following is the syntax.

```
autoResetEvent.WaitOne(TimeSpan.FromSeconds(2))
```

Set Method

The Set method sent the signal to the waiting thread to proceed with its work. Following is the syntax to call the Set method.

```
autoResetEvent.Set();
```

Note: The most important point that you need to remember is both threads will share the same AutoResetEvent object. Any thread can enter into a wait state by calling the WaitOne() method of the AutoResetEvent object. When the other thread calls the Set() method it unblocks the waiting thread.

Example to Understand AutoResetEvent in C#:

Let us see an example to understand the AutoResetEvent in C#. In the below example, we have two threads. The Main thread will invoke the main method and the NewThread which will invoke the SomeMethod method. The main method will invoke the new thread and the new thread actually go and run the SomeMethod. And the SomeMethod will first print the first statement i.e. Starting..... and then it invokes the WaitOne() method which Put the current thread i.e.NewThread into the waiting state until it receives the signal. Then inside the static void Main method, when we press the enter key, it will invoke the Set method which will send a signal to other threads to resume their work i.e. send the signal to NewThread to resume its work, and the new thread then prints Finishing..... on the console window.

```
using System;
using System.Threading;
namespace SemaphoreDemo
{
    class Program
    {
        static AutoResetEvent autoResetEvent = new AutoResetEvent(false);

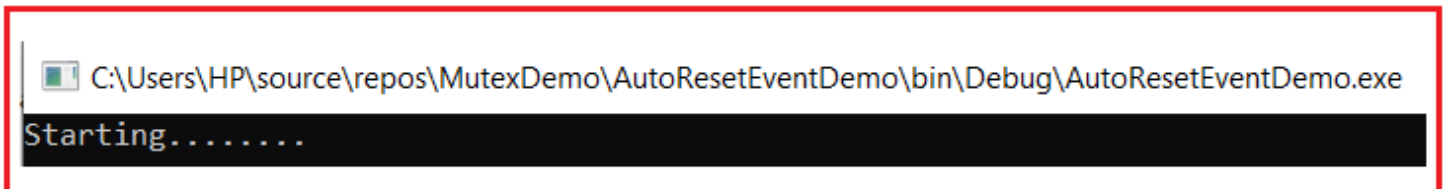
        static void Main(string[] args)
        {
            Thread newThread = new Thread(SomeMethod)
            {
                Name = "NewThread"
            };
            newThread.Start(); //It will invoke the SomeMethod in a different
            //To See how the SomeMethod goes in halt mode
            //Once we enter any key it will call set method and the SomeMethod
            Resume its work
        }
    }
}
```

```

        Console.ReadLine();
        //It will send a signal to other threads to resume their work
        autoResetEvent.Set();
    }
    static void SomeMethod()
    {
        Console.WriteLine("Starting.....");
        //Put the current thread into waiting state until it receives the
        autoResetEvent.WaitOne(); //It will make the thread in halt mode
        Console.WriteLine("Finishing.....");
        Console.ReadLine(); //To see the output in the console
    }
}

```

Now, run the application and you will get the following message.

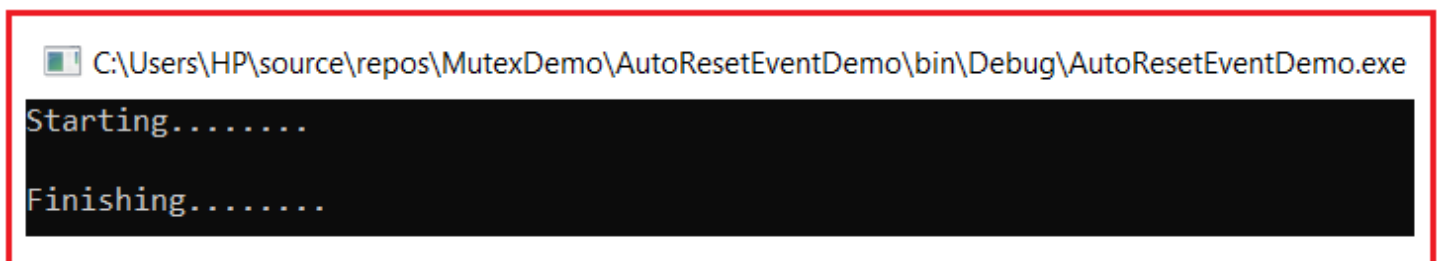


```

C:\Users\HP\source\repos\MutexDemo\AutoResetEventDemo\bin\Debug\AutoResetEventDemo.exe
Starting.....

```

At this point, the main thread is called the New Thread and the new thread executes the first statement i.e. printing the first message on the console and then invoking the WaitOne method. Once it invokes the WaitOne method the new thread goes into the sleep state. Next, when we press the enter key, then the main method invokes the Set method which will send a signal to other threads to resume their work. At this point, SomeMethod will resume its work and continue and you will see the Finishing message on the console window as shown below.

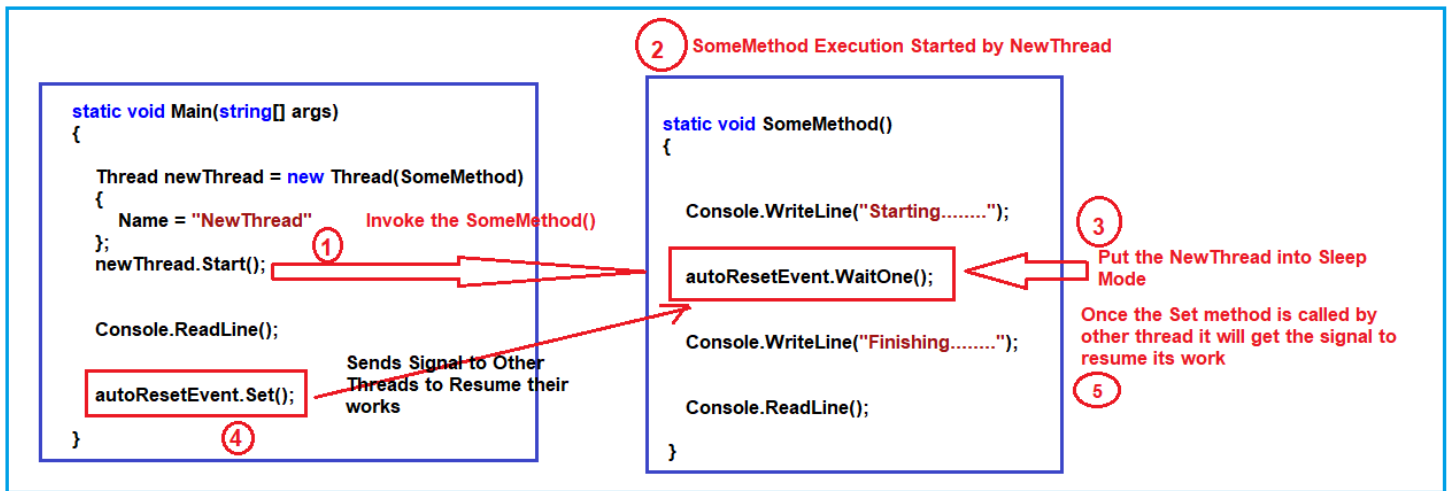


```

C:\Users\HP\source\repos\MutexDemo\AutoResetEventDemo\bin\Debug\AutoResetEventDemo.exe
Starting.....
Finishing.....

```

For a better understanding of the above program workflow please have a look at the below image.



Note: There is no guarantee that every call to the Set method will release a thread. If two calls are too close together, so that the second call occurs before a thread has been released, only one thread is released. It's as if the second call did not happen. Also, if Set is called when there are no threads waiting and the AutoResetEvent is already signaled, the call has no effect.

ManualResetEvent Class in C#:

The ManualResetEvent Class in C# works exactly the same as the AutoResetEvent Class in C#. Let us rewrite the same example using ManualResetEvent and then we will discuss the differences between them. Simply, replace the AutoResetEvent class with ManualResetEvent class in the below example.

```

using System;
using System.Threading;
namespace SemaphoreDemo
{
    class Program
    {
        static ManualResetEvent manualResetEvent = new ManualResetEvent(false);
        static void Main(string[] args)
        {
            Thread newThread = new Thread(SomeMethod)
            {
                Name = "NewThread"
            };
            newThread.Start(); //It will invoke the SomeMethod in a different
            //To See how the SomeMethod goes in halt mode
            //Once we enter any key it will call set method and the SomeMethod
            Resume its work
            Console.ReadLine();
            //It will send a signal to other threads to resume their work
            manualResetEvent.Set();
        }
        static void SomeMethod()
        {
            Console.WriteLine("Starting.....");
            //Put the current thread into waiting state until it receives the

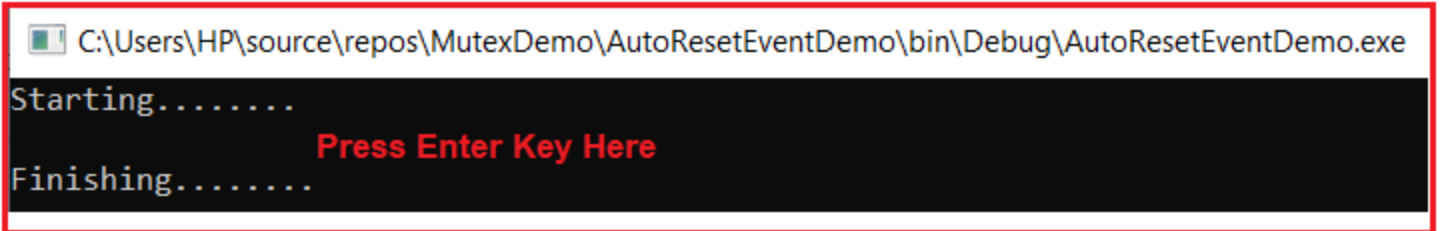
```

```

        manualResetEvent.WaitOne(); //It will make the thread in halt mode
        Console.WriteLine("Finishing.....");
        Console.ReadLine(); //To see the output in the console
    }
}
}

```

Output:



```

C:\Users\HP\source\repos\MutexDemo\AutoResetEventDemo\bin\Debug\AutoResetEventDemo.exe
Starting.....
Press Enter Key Here
Finishing.....

```

What are the Differences between AutoResetEvent and ManualResetEvent in C#?

Let us understand the differences with some examples. In AutoResetEvent, for every WaitOne method, there should be a Set method. That means if we are using the WaitOne method 2 times, then we should use the Set method 2 times. If we use the Set method 1 time, then the 2nd WaitOne method will be hung in the waiting state and will not be released. To understand this better please have a look at the below example.

```

using System;
using System.Threading;
namespace SemaphoreDemo
{
    class Program
    {
        static AutoResetEvent manualResetEvent = new AutoResetEvent(false);
        static void Main(string[] args)
        {
            Thread newThread = new Thread(SomeMethod)
            {
                Name = "NewThread"
            };
            newThread.Start(); //It will invoke the SomeMethod in a different
            //To See how the SomeMethod goes in halt state let sleep the main
            3 secs
            Thread.Sleep(3000);
            Console.WriteLine("Releasing the WaitOne 1 by Set 1");
            manualResetEvent.Set(); //Set 1 will relase the Wait 1
            //To See how the SomeMethod goes in halt state let sleep the main
            3 secs
            Thread.Sleep(5000);
            Console.WriteLine("Releasing the WaitOne 2 by Set 2");
            manualResetEvent.Set(); //Set 2 will relase the Wait 2
            Console.ReadKey();
        }
    }
}

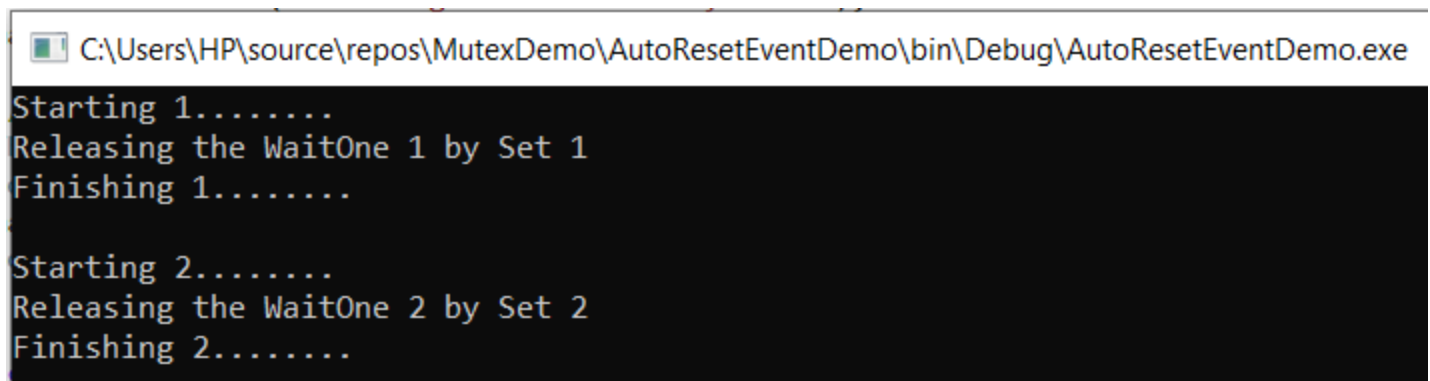
```

```

static void SomeMethod()
{
    Console.WriteLine("Starting 1.....");
    manualResetEvent.WaitOne(); //Wait 1
    Console.WriteLine("Finishing 1.....");
    Console.WriteLine();
    Console.WriteLine("Starting 2.....");
    manualResetEvent.WaitOne(); //Wait 2
    Console.WriteLine("Finishing 2.....");
}
}
}

```

Output:



```

C:\Users\HP\source\repos\MutexDemo\AutoResetEventDemo\bin\Debug\AutoResetEventDemo.exe
Starting 1.....
Releasing the WaitOne 1 by Set 1
Finishing 1.....

Starting 2.....
Releasing the WaitOne 2 by Set 2
Finishing 2.....

```

For each WaitOne method, we should and must have a Set method if we are using AutoResetEvent in C#. If we have two WaitOne methods and we have one Set method, then the second WaitOne method will hang in the sleep mode and will not release. For a better understanding, please have a look at the below example.

```

using System;
using System.Threading;
namespace SemaphoreDemo
{
    class Program
    {
        static AutoResetEvent manualResetEvent = new AutoResetEvent(false);
        static void Main(string[] args)
        {
            Thread newThread = new Thread(SomeMethod)
            {
                Name = "NewThread"
            };
            newThread.Start(); //It will invoke the SomeMethod in a different
            //To See how the SomeMethod goes in halt state let sleep the main
            Thread.Sleep(3000);
            Console.WriteLine("Releasing the WaitOne 1 by Set 1");
            manualResetEvent.Set(); //Set 1 will relase the Wait 1
        }
    }
}

```

3 secs


```

        Console.ReadKey();
    }
    static void SomeMethod()
    {
        Console.WriteLine("Starting 1.....");
        manualResetEvent.WaitOne(); //Wait 1
        Console.WriteLine("Finishing 1.....");
        Console.WriteLine();
        Console.WriteLine("Starting 2.....");
        manualResetEvent.WaitOne(); //Wait 2
        Console.WriteLine("Finishing 2.....");
    }
}

```

Output: Finishing 2..... statement will never be executed; you will get the following output.

```

Starting 1.....
Releasing the WaitOne 1 by Set 1
Finishing 1.....

Starting 2.....

```

But if we write the previous example using ManualResetEvent then it will work. That is one Set method in ManualResetEvent can release all the WaitOne methods. For a better understanding, please have a look at the below example.

```

using System;
using System.Threading;
namespace SemaphoreDemo
{
    class Program
    {
        static ManualResetEvent manualResetEvent = new ManualResetEvent(false);
        static void Main(string[] args)
        {
            Thread newThread = new Thread(SomeMethod)
            {
                Name = "NewThread"
            };
            newThread.Start(); //It will invoke the SomeMethod in a different
            //To See how the SomeMethod goes in halt state let sleep the main

            Thread.Sleep(3000);
            Console.WriteLine("Releasing the WaitOne 1 by Set 1");
            manualResetEvent.Set(); //Set will release all the WaitOne

            Console.ReadKey();
        }
        static void SomeMethod()
    }
}

```

3 secs

```

    {
        Console.WriteLine("Starting 1.....");
        manualResetEvent.WaitOne(); //Wait 1
        Console.WriteLine("Finishing 1.....");
        Console.WriteLine();
        Console.WriteLine("Starting 2.....");
        manualResetEvent.WaitOne(); //Wait 2
        Console.WriteLine("Finishing 2.....");
    }
}

```

Output:

```

Starting 1.....
Releasing the WaitOne 1 by Set 1
Finishing 1.....

Starting 2.....
Finishing 2.....

```

So, the one and the only difference between `AutoResetEvent` and `ManualResetEvent` in C# is that for each `WaitOne` method there should be a corresponding `Set` method in `AutoResetEvent` while for all the `WaitOne` methods, one `Set` method is enough to release in the case of `ManualResetEvent`.

In the next article, I am going to discuss [Thread Life Cycle in C#](#) with Examples. Here, in this article, I try to explain **AutoResetEvent and ManualResetEvent in C#** with Examples. I hope you enjoy this `AutoResetEvent and ManualResetEvent in C#` with Examples article.